

IN100 Initiation au Python

Contrôle continu 2

Étudiant :	Heure Début	Note (/5)
Correcteur :	Heure Fin	

Consignes:

Vous avez 20 minutes pour coder la solution. Seules les types et les instructions vues en cours sont autorisées. Tout usage d'une fonction venant d'une bibliothèque qui n'a pas été vue en cours ne sera pris en compte.

Sujet:

Un billet de loterie est donné par 6 nombres entiers compris entre 1 et 48.

1. En utilisant les notions vues en cours, donnez une fonction permettant de générer un billet quelconque pour un joueur. Nous rappelons que la fonction `random.randint(a,b)` permet de générer des nombres aléatoires entre `a` et `b`.

Exemple d'usage:

```
import random

print(random.randint(3,9))
```

2. On fixe le billet `B=[3,34,23,11,42,29]`. Un billet est gagnant s'il contient au moins trois nombres de `B`. Écrire une fonction permettant de vérifier, pour un billet quelconque, si celui-ci est gagnant. Si le billet est gagnant, votre fonction doit renvoyer tous les nombres que ce billet a en commun avec le billet `B`. Sinon, elle renverra `None`.

Remarques étudiant :
Remarques correcteur :

IN100 Initiation au Python

Contrôle continu 2

Étudiant :	Heure Début	Note (/5)
Correcteur :	Heure Fin	

Consignes:

Vous avez 20 minutes pour coder la solution. Seules les types et les instructions vues en cours sont autorisées. Tout usage d'une fonction venant d'une bibliothèque qui n'a pas été vue en cours ne sera pris en compte.

Sujet:

1. En utilisant une seule ligne de code, créez une liste contenant tous les entiers compris entre 1 et 7.

```
liste_initiale=[1,2,3,4,5,6,7]
```

2. Écrire une fonction qui prend en entrée une liste de taille quelconque, décale toutes les éléments de la liste de i positions vers la gauche, de manière circulaire. Ainsi premières i éléments se retrouvent à la fin de la liste.

Par exemple, pour la liste créée à la question 1 un décalage de 3 donnera la liste suivante:

```
liste_decallee=[4,5,6,7,1,2,3]
```

3. Écrire une fonctions qui prend en entrée une matrice de m lignes et n colonnes et modifie cette matrice comme suit: la première ligne est décalée de manière circulaire de 1, la deuxième ligne est décalée de manière circulaire de 2, ..., l'avant dernière ligne est décalée circulairement de $m - 1$ et la dernière ligne est inversée. La fonction devra renvoyer **None**.

Remarques étudiant :
Remarques correcteur :

IN100 Initiation au Python

Contrôle continu 2

Étudiant :	Heure Début	Note (/5)
Correcteur :	Heure Fin	

Consignes: Vous avez 20 minutes pour coder la solution. Seules les types et les instructions vues en cours sont autorisées. Tout usage d'une fonction venant d'une bibliothèque qui n'a pas été vue en cours ne sera pris en compte.

Sujet: On lance deux dés à six faces et on note leurs résultats a, b . Les différentes cas que l'on peut obtenir sont:

- PAIRE: deux dés identiques et un autre différent: ex 3,3 ou 4,4
- SUPER : une des deux valeurs est 6 (ex 2,6)
- SUPER EXTRAORDINAIRE les deux valeurs sont 6 (ex 6,6)
- NADA: dans tous les autres cas.

Les premières trois cas sont gagnants. Le but est d'écrire un programme qui simule le comportement décrit précédemment.

1. Créer une fonction `est_gagnant` qui prend en paramètre un tuple de 2 valeurs et renvoie `True` si le tuple est gagnant, `False` sinon.
2. Écrire une fonction qui génère plusieurs tuples de nombres aléatoires entre 1 et 6, en utilisant la fonction `random.randint`, et qui s'arrête lorsque un tuple gagnant est trouvé. La fonction renverra le message associé au tuple gagnant et le nombre de fois les dés ont été "joués". Elle utilisera la fonction de la question 1.

Exemple d'utilisation :

```
import random
print(random.randint(2, 5)) # affichera une valeur entre 2 et 5 inclus
```

Remarques étudiant :

Remarques correcteur :

IN100 Initiation au Python

Contrôle continu 2

Étudiant :	Heure Début	Note (/5)
Correcteur :	Heure Fin	

Consignes:

Vous avez 20 minutes pour coder la solution. Seules les types et les instructions vues en cours sont autorisées. Tout usage d'une fonction venant d'une bibliothèque qui n'a pas été vue en cours ne sera pris en compte.

Sujet:

Afin de s'entraîner pour une compétition sportive, Arthur doit pratiquer plusieurs activités sportives par jour, pendant n jours. On utilise une liste imbriquée pour représenter les activités qu'il pratique chaque jour. Par exemple pour $n = 5$ jours, on a:

```
planning_activités=[["jogging", "yoga", "natation"], [], ["natation", "tennis"], ["tennis"], ["natation", "badminton"]].
```

Cela veut dire que le jour 1 il a pratiqué le jogging, le yoga et la natation, le jour 2 il n'a pratiqué aucune activité etc.

1. Écrire une fonction `nombre_jours_activites` qui prend en argument le planning et une activité et renvoie le nombre de jours pendant lesquels cette activité a été pratiquée.
2. Écrire une fonction `total_activites` qui prend en argument le planning et renvoie le nombre total d'activités pratiquées sur toute la période.

Remarques étudiant :
Remarques correcteur :

IN100 Initiation au Python

Contrôle continu 2

Étudiant :	Heure Début	Note (/5)
Correcteur :	Heure Fin	

Consignes:

Vous avez 20 minutes pour coder la solution. Seules les types et les instructions vues en cours sont autorisées. Tout usage d'une fonction venant d'une bibliothèque qui n'a pas été vue en cours ne sera pris en compte.

Sujet

Une file est une structure de donnée basé sur le principe que les premiers éléments ajoutés dans la file seront les premiers à en être retirés. Dans cet exercice, nous utiliserons une liste pour représenter une file pour gérer les patients dans une clinique. Chaque patient est représenté par un entier correspondant au temps estimé de consultation en minutes.

1. Écrire une fonction qui prend en argument une file et un entier, et qui rajoute cet élément dans la file, donc en début de liste. Vous utiliserez cette fonction pour rajouter dans votre file les temps de consultation suivants:

15
20
15
30
25

2. Écrire une fonction qui prend en argument la liste donnant la file et qui enlève le dernier élément de la liste et renvoie cet élément.
3. Écrire une fonction `traiter_patients` prend en argument la file de patients, traite tous les patients dans la file jusqu'à ce qu'elle soit vide et renvoie le temps total passé à consulter tous les patients.

Remarques étudiant :
Remarques correcteur :